

API Security Risk Analysis Report

Future Interns – Cyber Security Internship
Task 3: API Security Risk Analysis

Prepared by: Mohammed Naseeh Syed
Institution: City Colleges of Chicago
Date: 03 March 2026

Executive Summary

This report presents a read-only API security assessment conducted on the public testing API JSONPlaceholder. The objective of the analysis was to observe potential API security risks such as unauthenticated access, excessive data exposure, and weak access control behaviors.

Testing was performed using Postman and web browser inspection. Only safe GET requests were executed. No exploitation, bypass attempts or harmful actions were conducted during the assessment.

The analysis identified several common API security concerns that could become critical in real production systems. These include open endpoints, exposure of user data, lack of authentication, and the absence of visible rate-limiting controls.

Scope of Assessment

Target API

<https://jsonplaceholder.typicode.com>

Testing Type

Read-only API security review

Tools Used

- Postman
- Google Chrome
- Microsoft Word / PDF
- GitHub

Testing Methodology

- Endpoint inspection
 - Response analysis
 - Header inspection
 - Browser accessibility testing
 - Documentation of potential risks
-

Methodology

The following methodology was used during the security review:

1. Selected a publicly available demo API.
2. Performed safe **GET requests** using Postman.
3. Inspected API responses and returned data fields.
4. Observed authentication and authorization behavior.
5. Reviewed response headers for security controls.
6. Tested browser access to API endpoints.
7. Documented identified risks and recommendations.

API Endpoints Tested

Endpoint	Description
<code>/users</code>	Returns a list of users
<code>/users/1</code>	Returns specific user data
<code>/users/2</code>	Returns another user record
<code>/posts</code>	Returns post content
<code>/comments</code>	Returns comments including email data

Security Findings

ID	Risk	Severity	Observation	Recommendation
1	Unauthenticated endpoint access	Medium	<code>/users</code> accessible without authentication	Implement authentication for sensitive endpoints
2	Excessive data exposure	Medium	User records expose email, phone, address, geo data	Return only required fields
3	Potential object-level authorization weakness	High (contextual)	Changing <code>/users/1</code> to <code>/users/2</code> returns another user record	Implement strict server-side authorization
4	Open posts endpoint	Medium	<code>/posts</code> accessible without authentication	Apply proper access control
5	Email exposure in comments	Medium	<code>/comments</code> reveals email addresses	Mask or restrict sensitive fields

6	No visible rate-limiting evidence observed	Medium	Header inspection showed no obvious request throttling	Implement API gateway rate limiting
---	--	--------	--	-------------------------------------

Detailed Findings

Finding 1 – Unauthenticated Endpoint Access

The `/users` endpoint was accessible without any authentication mechanism such as API keys or tokens. In real-world applications, this behavior may expose sensitive user information.

Risk: Unauthorized data access

Recommendation: Require authentication and authorization checks.

Finding 2 – Excessive Data Exposure

The `/users` endpoint returned detailed information including:

- Email
- Phone number
- Address
- Geolocation
- Company details

In production APIs, returning excessive data increases privacy risks and potential misuse.

Recommendation: Implement response filtering and return only necessary fields.

Finding 3 – Potential Object Level Authorization Weakness

By modifying the URL from `/users/1` to `/users/2`, another user record was retrieved successfully without additional access validation.

In real systems, this behavior could allow unauthorized access to other users' data.

Recommendation: Enforce server-side authorization checks for each object request.

Finding 4 – Open Content Endpoint

The `/posts` endpoint was publicly accessible and returned content data without authentication.

While acceptable for demo APIs, production systems should ensure that only authorized users can access protected resources.

Recommendation: Apply authentication and role-based access control.

Finding 5 – Email Exposure in Comments

The `/comments` endpoint returned email values in its response. This could potentially be exploited for phishing, spam, or user profiling.

Recommendation: Remove or mask sensitive user data.

Finding 6 – Header Inspection

Response headers were inspected in Postman. Standard headers such as:

- Content-Type
- Cache-Control
- Content-Length
- ETag

were present. However, no visible rate-limiting headers were observed during basic inspection.

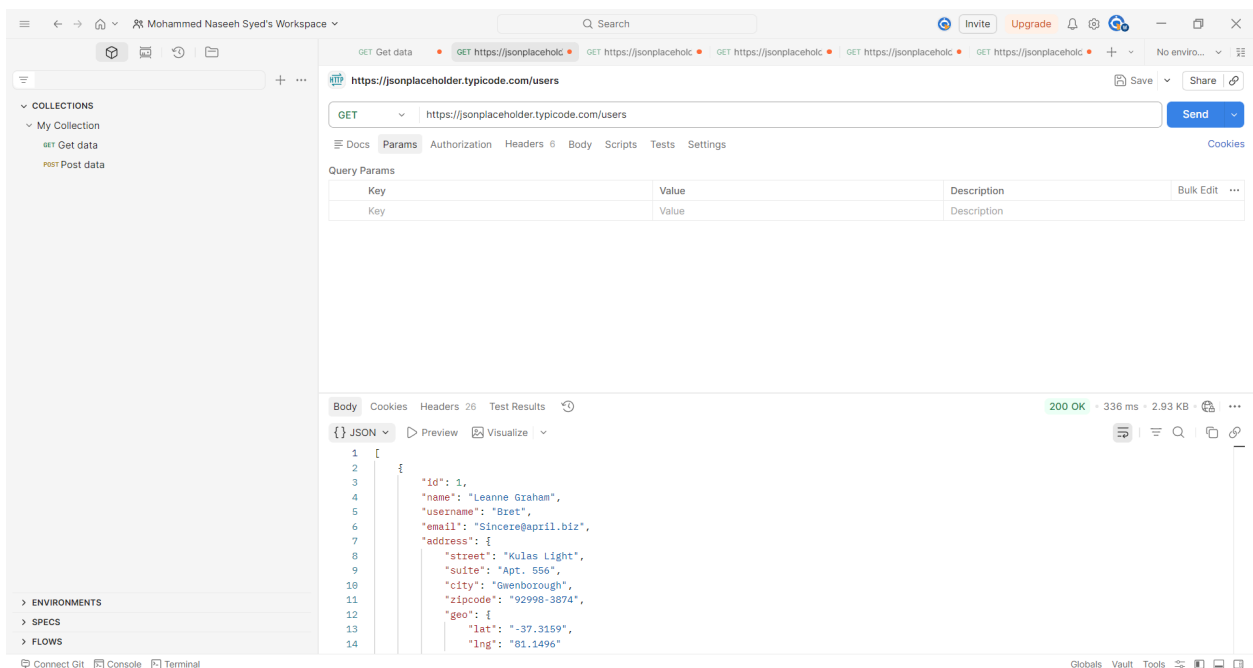
Risk: Potential API abuse through automated requests.

Recommendation: Implement request throttling and API monitoring.

Evidence Screenshots

The following screenshots demonstrate the testing performed during the analysis.

Screenshot 1 – Users Endpoint



Screenshot 2 – User ID 1 Endpoint

Screenshot of a REST client interface showing a GET request to `https://jsonplaceholder.typicode.com/users/1`. The response is a JSON object representing a user with ID 1.

Request: GET `https://jsonplaceholder.typicode.com/users/1`

Response: 200 OK - 61 ms - 1.39 KB

```
{
  "id": 1,
  "name": "Leanne Graham",
  "username": "Bret",
  "email": "Sincere@april.biz",
  "address": {
    "street": "Kulas Light",
    "suite": "Apt. 556",
    "city": "Gwenborough",
    "zipcode": "92998-3874",
    "geo": {
      "lat": "-37.3159",
      "lng": "81.1496"
    }
  }
}
```

Screenshot 3 – User ID 2 Endpoint

Screenshot of a REST client interface showing a GET request to `https://jsonplaceholder.typicode.com/users/2`. The response is a JSON object representing a user with ID 2.

Request: GET `https://jsonplaceholder.typicode.com/users/2`

Response: 200 OK - 158 ms - 1.39 KB

```
{
  "id": 2,
  "name": "Ervin Howell",
  "username": "Antonette",
  "email": "Shanna@melissa.tv",
  "address": {
    "street": "Victor Plains",
    "suite": "Suite 879",
    "city": "Wisokyburgh",
    "zipcode": "90566-7771",
    "geo": {
      "lat": "-43.9509",
      "lng": "-34.4618"
    }
  }
}
```

Screenshot 4 – Posts Endpoint

GET <https://jsonplaceholder.typicode.com/posts> <https://jsonplaceholder.typicode.com/users/2> [Save](#) [Share](#)

GET <https://jsonplaceholder.typicode.com/posts> [Send](#)

Docs Params Authorization Headers 6 Body Scripts Tests Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body Cookies Headers 24 Test Results [200 OK](#) · 162 ms · 7.98 KB [Copy](#) [Print](#) [Share](#)

[JSON](#) [Preview](#) [Visualize](#)

```
1 [
2   {
3     "userId": 1,
4     "id": 1,
5     "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
6     "body": "quia et suscipit\nsuscepit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto"
7   },
8   {
9     "userId": 1,
10    "id": 2,
11    "title": "qui est esse",
12    "body": "est rerum tempore vitae\nsequi sint nihil reprehenderit dolor beatae ea dolores neque\nfugiat blanditiis voluptate porro vel nihil molestiae ut reiciendis\nqui aperiam non debitis possimus qui neque nisi nulla"
13  }
14 ]
```

Connect Git Console Terminal

Globals Vault Tools [Copy](#) [Print](#) [Share](#)

Screenshot 5 – Comments Endpoint

GET <https://jsonplaceholder.typicode.com/comments> [Save](#) [Share](#)

GET <https://jsonplaceholder.typicode.com/comments> [Send](#)

Docs Params Authorization Headers 6 Body Scripts Tests Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body Cookies Headers 24 Test Results [200 OK](#) · 196 ms · 40.78 KB [Copy](#) [Print](#) [Share](#)

[JSON](#) [Preview](#) [Visualize](#)

```
1 [
2   {
3     "postId": 1,
4     "id": 1,
5     "name": "id labore ex et quam laborum",
6     "email": "Eliseo@gardner.biz",
7     "body": "laudantium enim quasi est quidem magnam voluptate ipsam eos\nntempora quo necessitatibus\ndolor quam autem quasi\nreiciendis et nam sapiente accusantium"
8   },
9   {
10    "postId": 1,
11    "id": 2,
12    "name": "quo vero reiciendis velit similique earum",
13    "email": "Jayne_Kuhic@sydney.com",
14  }
15 ]
```

Connect Git Console Terminal

Globals Vault Tools [Copy](#) [Print](#) [Share](#)

Screenshot 6 – Response Headers Inspection

The screenshot shows the Postman interface with a GET request to `https://jsonplaceholder.typicode.com/users`. The 'Headers' tab is selected, displaying a table of response headers. The status is 200 OK, and the response time is 336 ms.

Key	Value	Description
:status	200	
date	Mon, 09 Mar 2026 02:02:05 GMT	
content-type	application/json; charset=utf-8	
content-length	1847	
access-control-allow-credentials	true	
cache-control	max-age=43200	
content-encoding	gzip	
etag	W/"160d-1eMSsxeJRfnVLRBmYJSbcUZIqQ"	
expires	-1	

Screenshot 7 – Browser API Access

The screenshot shows a web browser displaying the JSON response from `https://jsonplaceholder.typicode.com/users`. The response is a JSON array of two user objects.

```
[{"id": 1, "name": "Leanne Graham", "username": "Bret", "email": "Sincere@april.biz", "address": {"street": "Kulas Light", "suite": "Apt. 556", "city": "Gwenborough", "zipcode": "92998-3874", "geo": {"lat": "-37.3159", "lng": "81.1496"}}, "phone": "1-770-736-8031 x56442", "website": "hildegard.org", "company": {"name": "Romaguera-Crona", "catchPhrase": "Multi-layered client-server neural-net", "bs": "harness real-time e-markets"}}, {"id": 2, "name": "Ervin Howell", "username": "Antonette", "email": "Shanna@melissa.tv", "address": {"street": "Victor Plains", "suite": "Suite 879", "city": "Wisokyburgh", "zipcode": "90566-7771", "geo": {"lat": "-43.9509", "lng": "-34.4618"}}, "phone": "010-692-6593 x99125", "website": "anastasia.net", "company": {}}
```

Conclusion

The JSONPlaceholder API is designed for testing purposes, but the assessment demonstrates several common API security concerns that could pose risks in real SaaS environments.

The most important security improvements for production APIs include:

- Authentication enforcement
- Response data minimization
- Object-level authorization checks
- Rate limiting and monitoring

Secure API design is essential to prevent unauthorized access, data leakage, and automated abuse.

Ethical Disclaimer

This security assessment was performed using **safe and ethical read-only testing methods** on a public demo API. No exploitation, bypass attempts or harmful actions were conducted during the analysis.